



FADO

Meating 1

Tiago Silva

CISUC · Data-Centric AI Lab

Department of Informatics Engineering, University of Coimbra



Outline

1. *Overall Assessment;*
2. *The Depth Problem;*

Overall Assessment

Strengths:

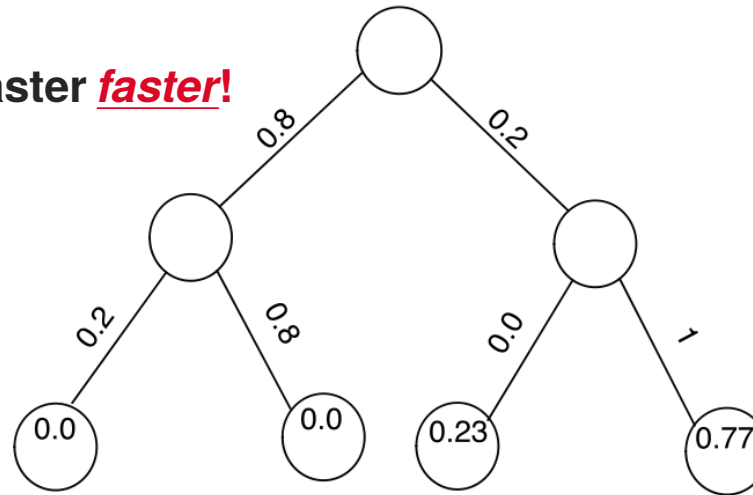
- ▶ Significant problem, well written/organized;
- ▶ Improves fairness metrics maintaining predictive performance;
- ▶ Rigorous statistical evaluation, reproducible code.

Weaknesses:

- ▶ Doubts about novelty - Lack of Differentiation from prior work;
- ▶ Limited Theoretical Justification of parameters (Lack of Ablation Studies);
- ▶ Should Ablate the effect of each component of the proposed method;
- ▶ More datasets.

Testing deeper trees with limited horsepower

To ablate tree depth we must be faster faster!



$$\text{MASK} = \begin{vmatrix} 1 & 1 & -1 & -1 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \end{vmatrix}$$

Update's Time Complexity:

$$O(B(2^n * 2^{(n-1)})) \sim O(B \cdot 4^d)$$

A prediction is the sum of the products of the node decisions along the path to each leaf.

Tree depth: mask products vs. recursive propagation

Before: mask approach

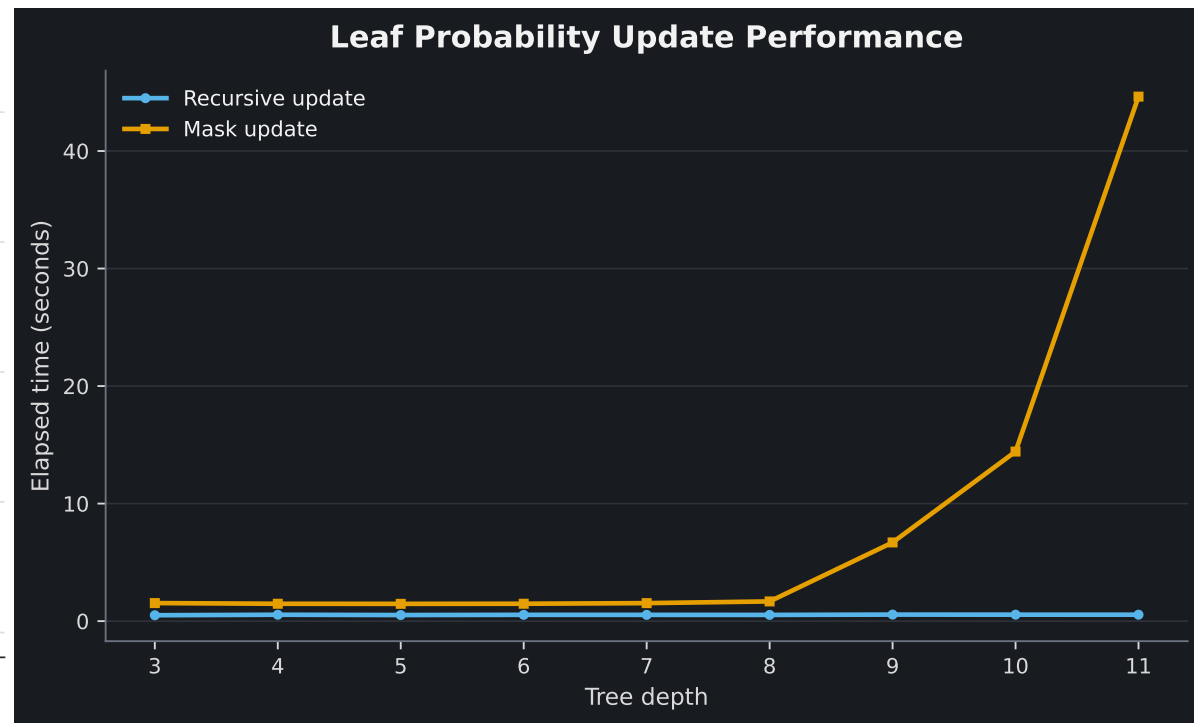
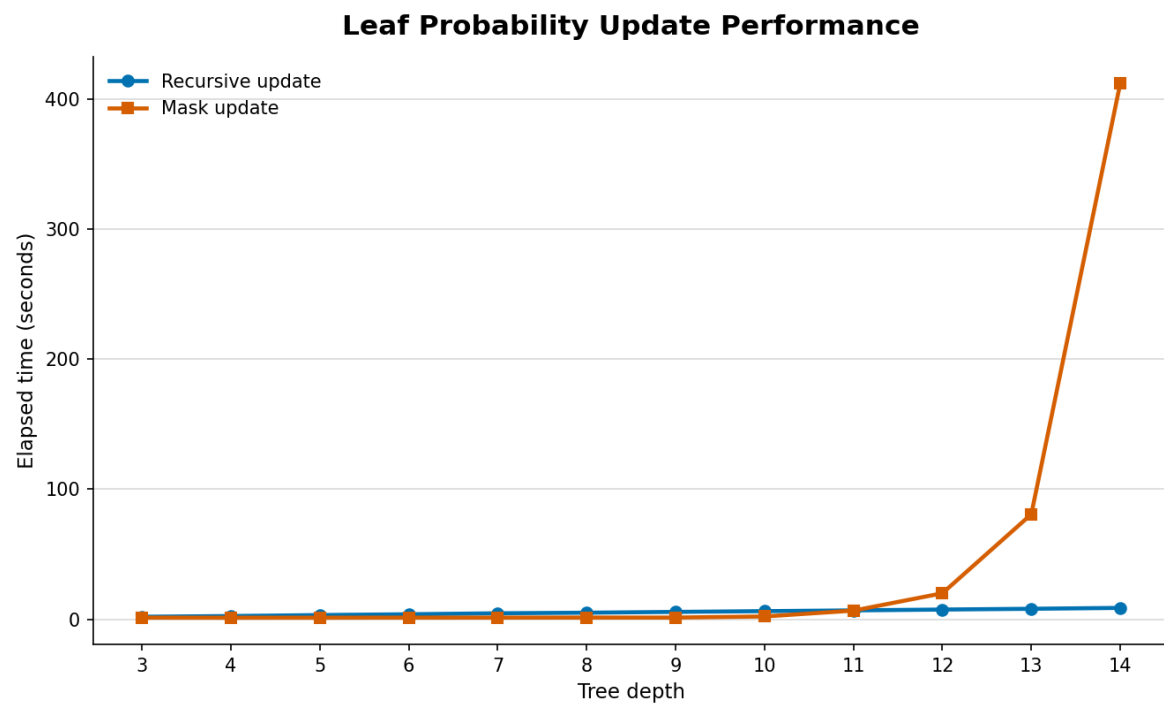
```
logits ← (inputs × weight + bias) / temperature
node_decisions ← activation(logits)
y ← expand_dims(node_decisions)
y_repeated ← repeat(y, num_leaves)
z ← y_repeated ⊙ mask
probs ← relu(z) + (ones - relu(-z)) + mask + ε
for each leaf  $\ell$ :
    leaf_probs[ $\ell$ ] ← product(probs[:, :,  $\ell$ ])
    # materialise every node-leaf pair
    # repeat the product for every leaf
prediction ← leaf_probs ×  $\theta$ 
```

After: recursive updater

```
logits ← (inputs × weight + bias) / temperature
node_decisions ← activation(logits)
root_prob ← 1
leaf_probs ← traverse(root, root_prob,
    node_decisions, depth)
    # propagate only to valid child branches
    # reuse each partial path probability
prediction ← leaf_probs ×  $\theta$ 
```

Complexity goes from $O(B \times 4^n)$ to $O(B \times 2^n)$, where n is the tree depth.

Tree depth: mask products vs. recursive performance



We can now ablate how trees' depth affect the forest's predictive and fairness metrics

¹Stopped at $n = 14$ for GPU and $n = 11$ for CPU because the mask's runtime became unbearable.

Rolling fairness metrics: incremental counters

Legacy recomputation

Each sample rescans the complete fairness window for DP and EO.

Complexity: $O(NW)$, where W is the window size.

Incremental window

Add the arriving sample and remove the expired sample from group counters.

Complexity: $O(NA)$, where A is the number of groups.

Benchmark

50,000 samples; window size 1,000; binary protected attribute.

15.39× faster with identical DP and EO at every timestep.

The counter-based window preserves the metric values while avoiding repeated scans of the same samples.

Real *impact* on model's *performance*

For an adult dataset run (32k samples) we were able to reduce from 37.12 minutes to 13.22 minutes.

We can now proceed to ablate tree depth!